



AuSRoS

Australian School of Robotic Systems

Lectorial D2 @ AuSRoS 2025

Robot Learning – Point Cloud Learning

Peyman Moghadam

July 2025

Australia's National Science Agency





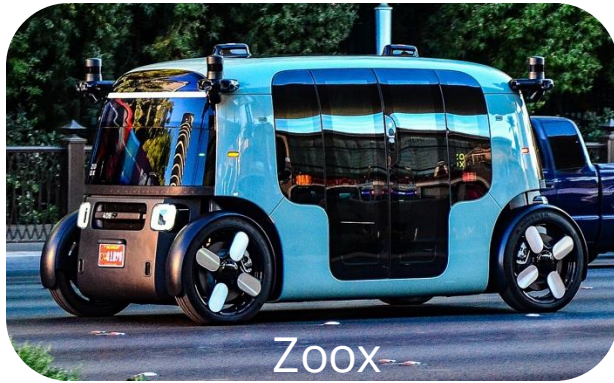
I would like to begin by acknowledging the Turrbal and Yaggera people as the Traditional Owners of the land that I am presenting on and pay my respect to their Elders past and present.



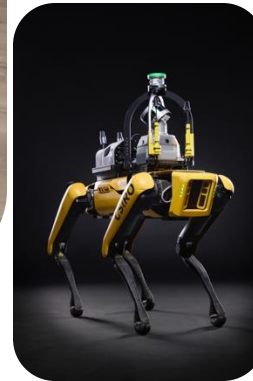
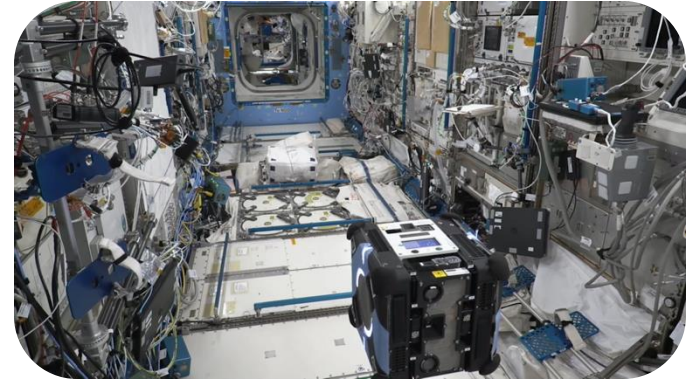
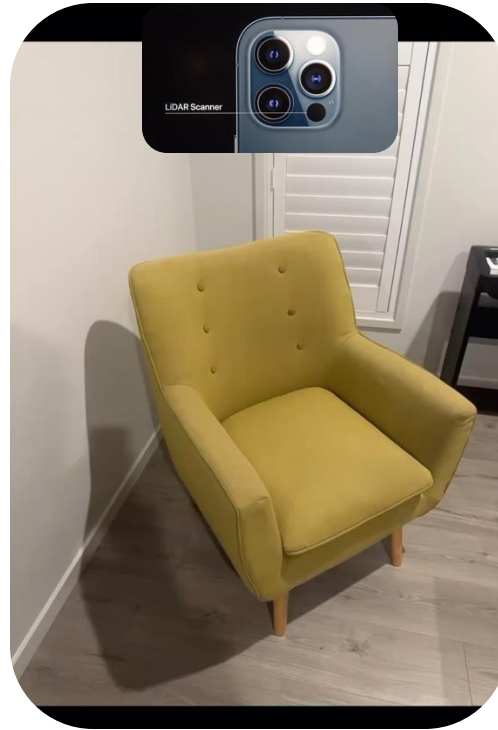
Outlines

- 3D Understanding Tasks
- Common 3D Sensors, 3D Scene Representations
- Sparse Convolution vs Dense Convolution
- Robot Learning Deep Dive: Lidar Place Recognition
- Challenge for Robot Learning
 - Domain Gaps
 - Catastrophic Forgetting
 - Benchmarks and Datasets

3D Data is Everywhere



Self-Driving cars

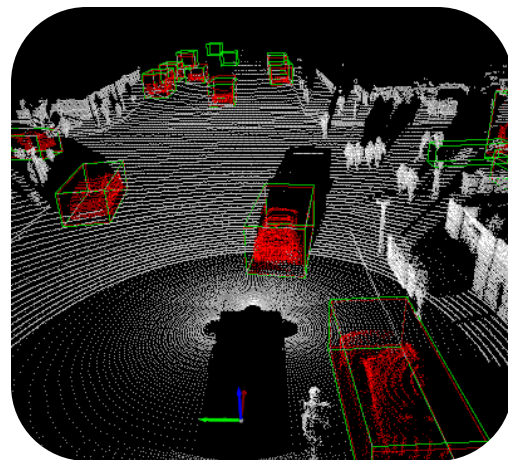


Robotics

3D Understanding Tasks



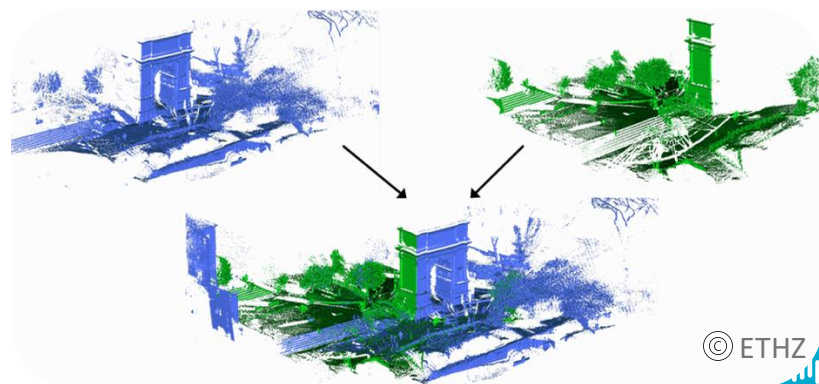
3D Semantic Segmentation



3D Object Detection & Tracking



3D Place Recognition



Point Cloud Registration

How 3D Data are Generated

RGB-D Sensors



Intel RealSense



CSIRO MRSS

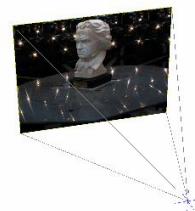
LiDAR (Light Detection and Ranging)



Zebedee



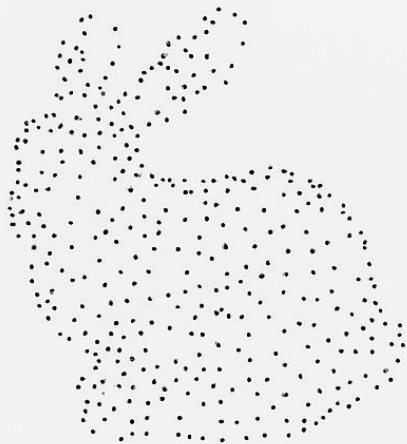
Multi-view Geometry



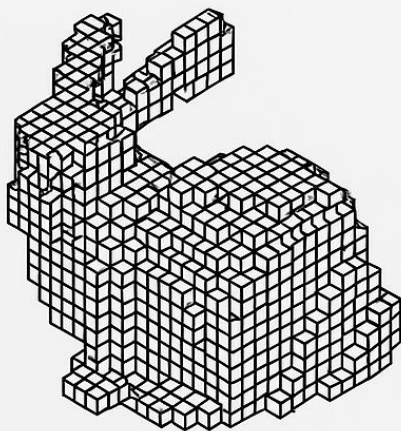
© CVG TUM

Common 3D Scene Representations (Explicit)

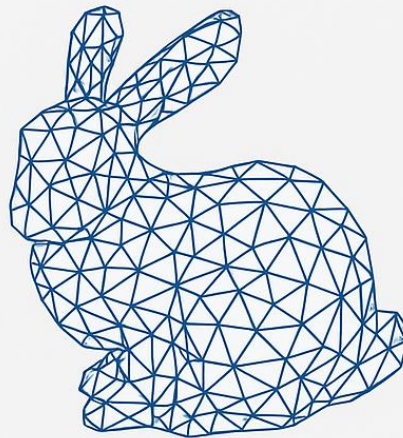
Point Cloud



Voxel

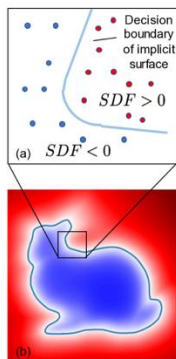


Mesh



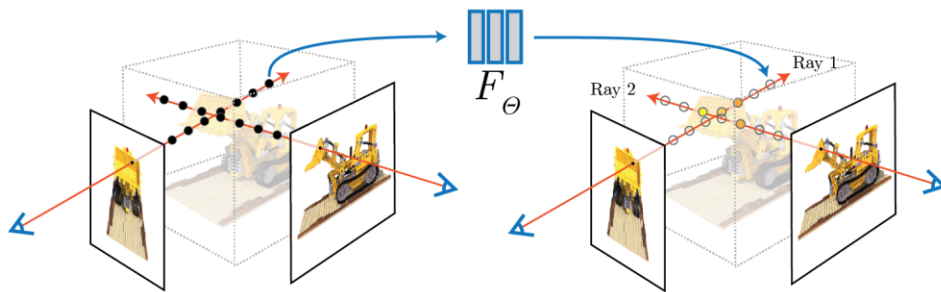
Common 3D Scene Representations (Implicit)

Neural Fields (*e.g.*, DeepSDF)



© Park et. al. 2019

Neural Fields (*e.g.*, Neural Radiance Fields)



© Mildenhall et al. 2021

3D Point Cloud Learning (from LiDAR)

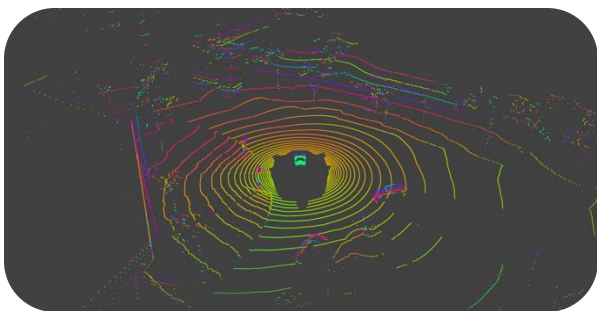


LiDAR



Point Cloud

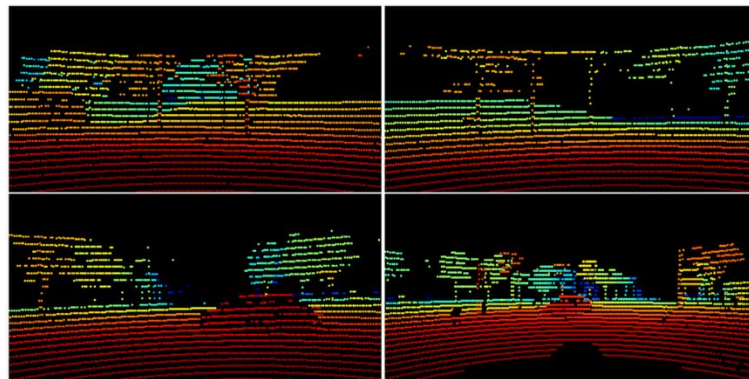
3D to 2D Projection



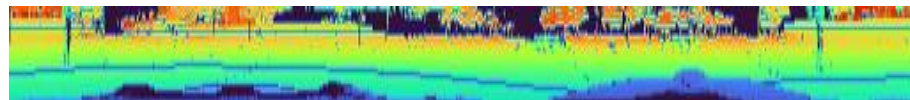
Point Cloud



Perspective
Projection



Range
Projection



0

2π

$$\begin{pmatrix} u \\ v \end{pmatrix} = \begin{pmatrix} \frac{1}{2} \left(1 - \frac{\arctan(y, x)}{\pi} \right) W \\ \left(1 - \frac{\arcsin\left(\frac{z}{r}\right) + |f_{\text{down}}|}{|f_{\text{down}}| + |f_{\text{up}}|} \right) H \end{pmatrix},$$

$$(u, v)^{\top} = \mathcal{P}(\mathbf{K} \mathbf{T}_{C \leftarrow L} [x, y, z, 1]^{\top})$$

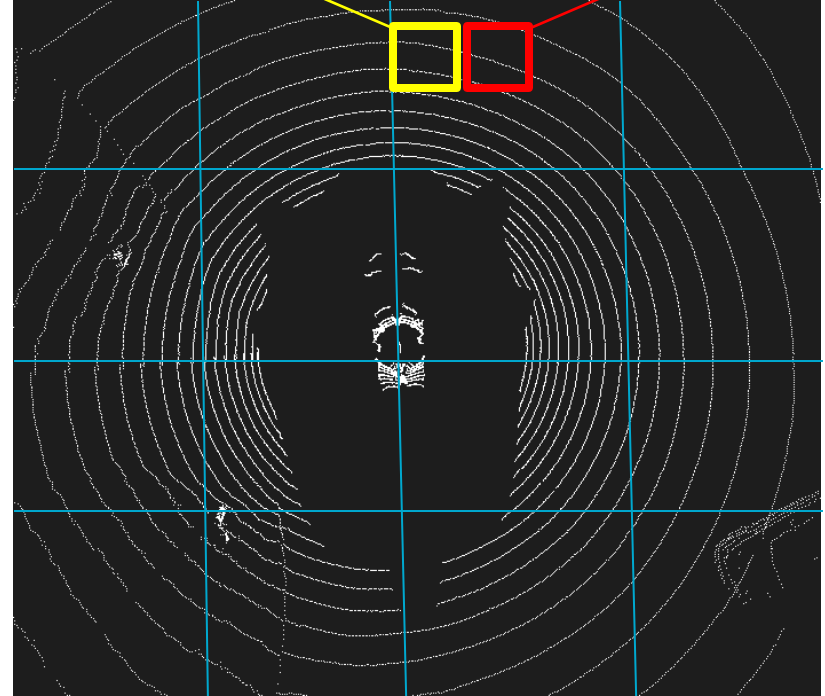
$$\mathcal{P}([x, y, z]^{\top}) = \frac{1}{z} [x, y]^{\top}, \quad \mathbf{T}_{C \leftarrow L} = \begin{bmatrix} \mathbf{R} & \mathbf{t} \\ \mathbf{0}^{\top} & 1 \end{bmatrix}$$

Challenges with 3D Point Clouds

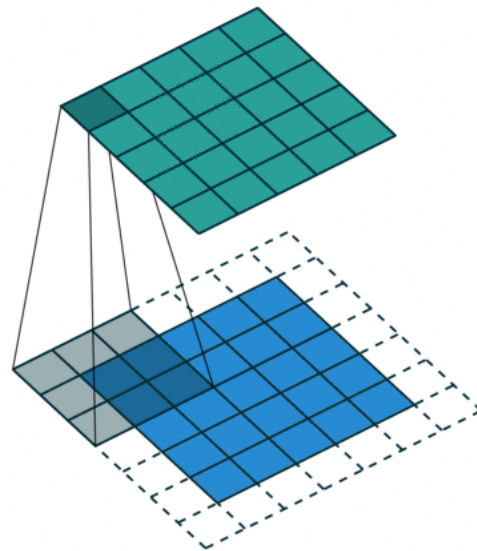
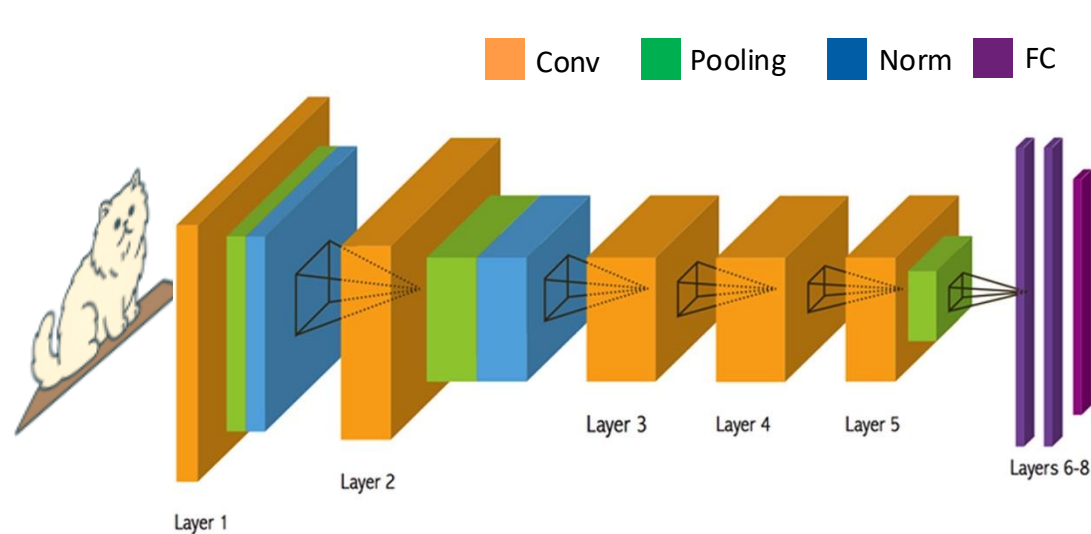
- **Sparse** - The points are not evenly sampled in different regions.
- **Unstructured** - Unlike images, point clouds are not placed on a grid. Each point is sampled independently, and its distance to its neighbours varies.
- **Invariant to permutations** - The order in which the points are stored does not alter the represented scene.

$X = 2.38$
 $Y = 5.73$
 $Z = -1.65$

$X = 2.74$
 $Y = 6.25$
 $Z = -1.64$

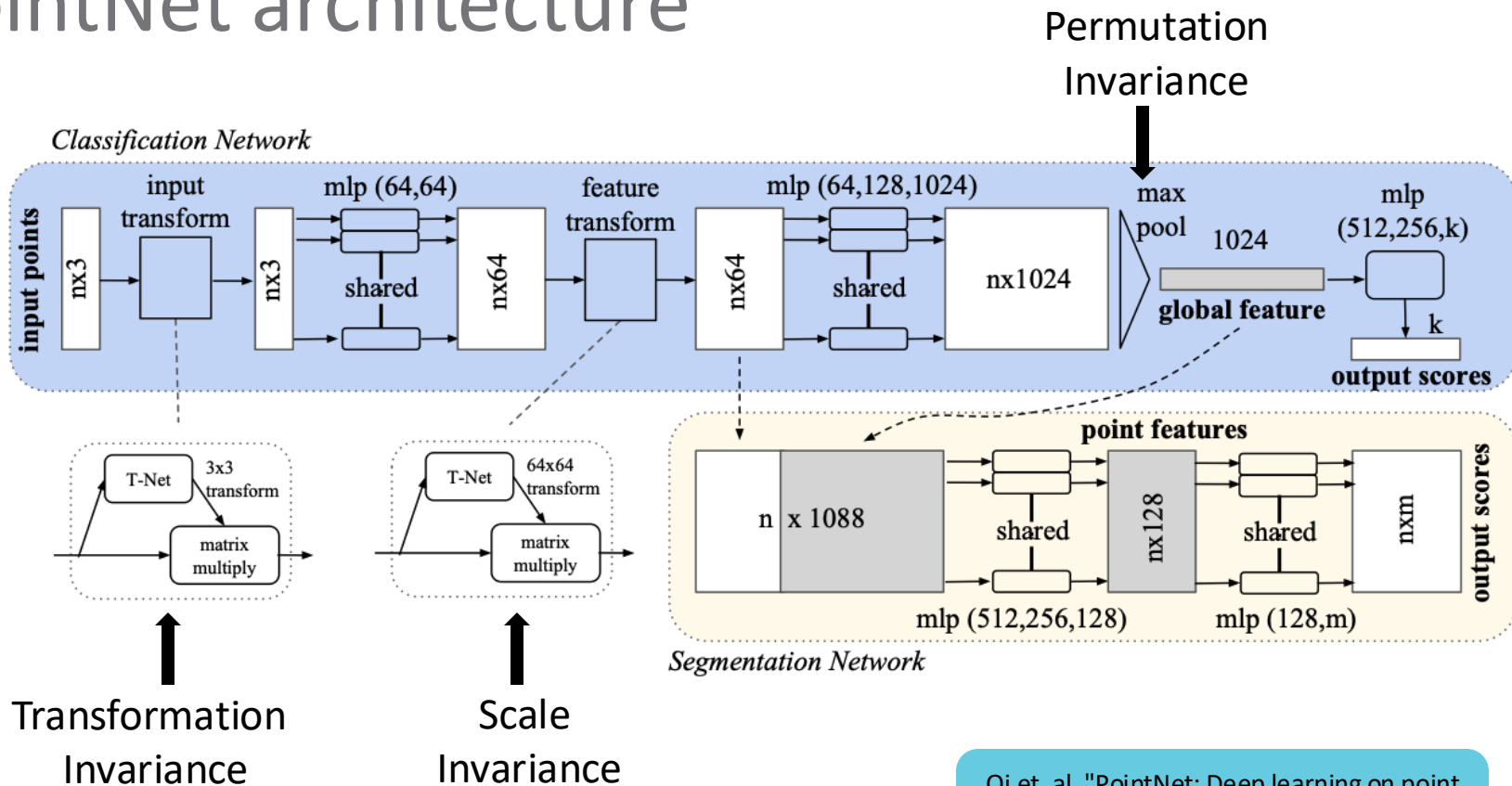


Recap: 2D CNNs Essentials



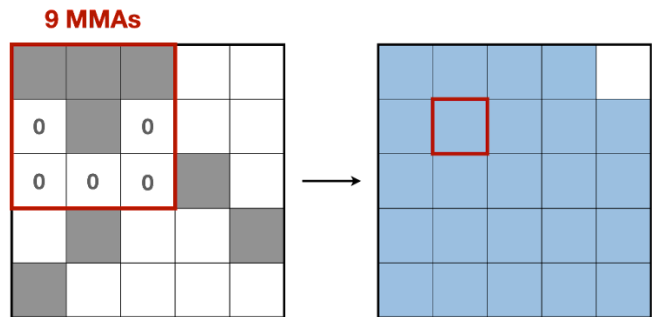
```
class torch.nn.Conv2d(  
    in_channels, out_channels, kernel_size=3,  
    stride=1, padding=1, dilation=1,  
    groups=1, bias=True, padding_mode='zeros')
```

PointNet architecture

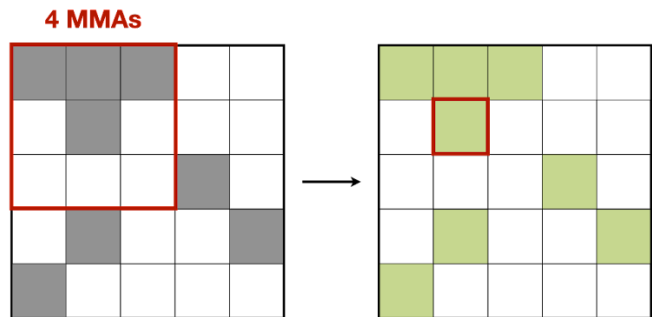


Qi et. al. "PointNet: Deep learning on point sets for 3D classification and segmentation (CVPR 2017)

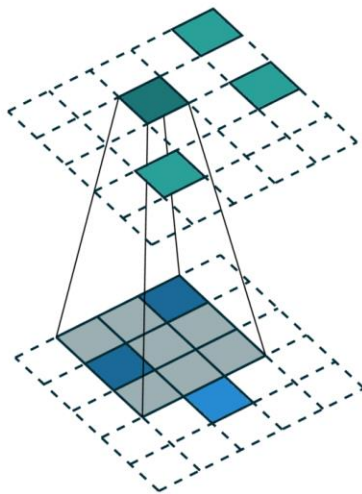
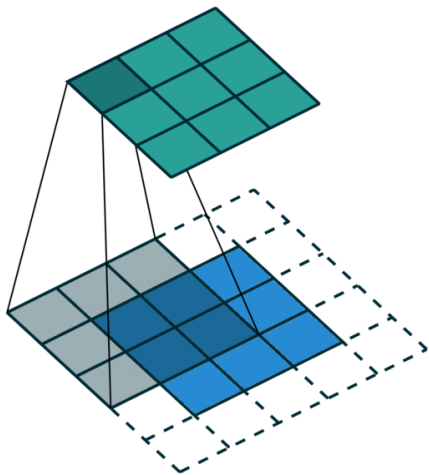
Neural Networks for Spatially Sparse Tensors



Dense Convolution



Sparse Convolution



TORCH SPARSE

© Chris Choy



Syntax is the same

```
import torch.nn as nn
class ConvBlock(nn.Sequential):
    def __init__(self,
                  in_channels: int,
                  out_channels: int,
                  kernel_size: Union[int, list, tuple],
                  stride: Union[int, list, tuple] = 1,
                  dilation: int = 1) -> None:
        super().__init__(
            nn.Conv2d(in_channels,
                      out_channels,
                      kernel_size,
                      stride=stride,
                      dilation=dilation),
            nn.BatchNorm2d(out_channels),
            nn.ReLU(True)
        )
```

Dense Convolution

```
import torchsparse.nn as spnn
class SparseConvBlock(nn.Sequential):
    def __init__(self,
                  in_channels: int,
                  out_channels: int,
                  kernel_size: Union[int, list, tuple],
                  stride: Union[int, list, tuple] = 1,
                  dilation: int = 1) -> None:
        super().__init__(
            spnn.Conv3d(in_channels,
                        out_channels,
                        kernel_size,
                        stride=stride,
                        dilation=dilation),
            spnn.BatchNorm(out_channels),
            spnn.ReLU(True)
        )
```

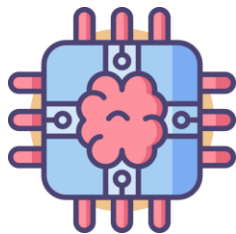
Sparse Convolution



Navigation



Locomotion



Learning methods



Manipulation



Localization

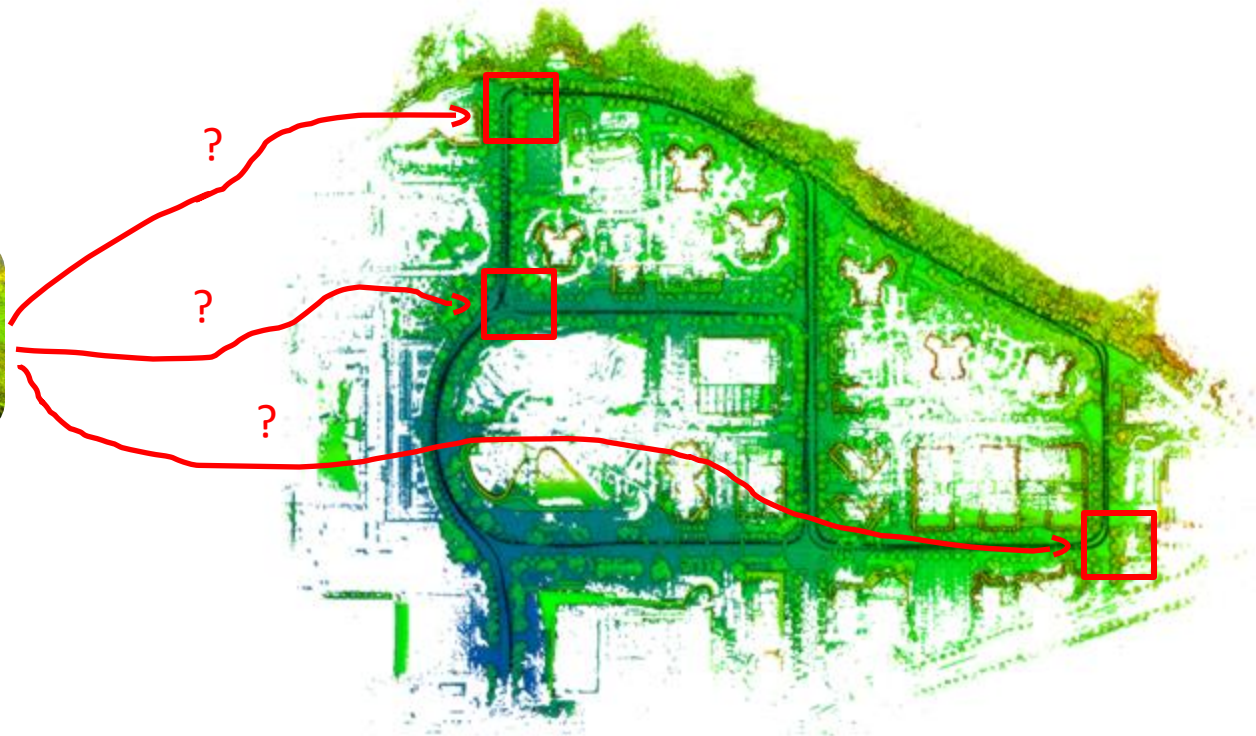
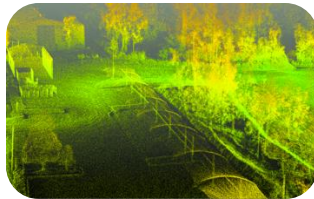
Lidar Place Recognition

What is Place Recognition

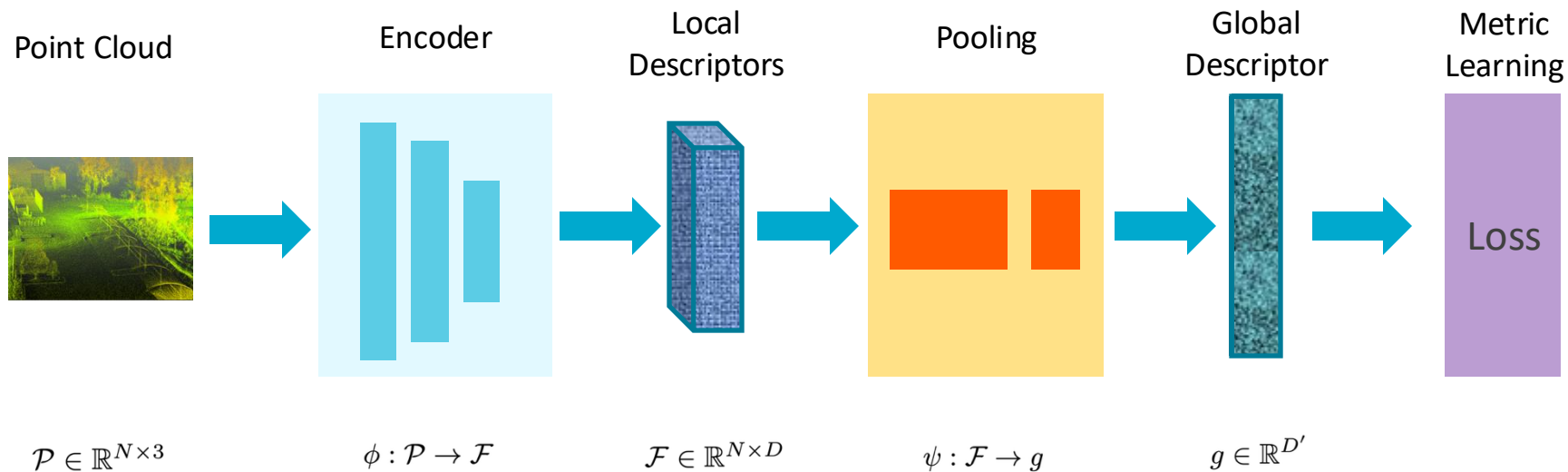
Lidar Place Recognition



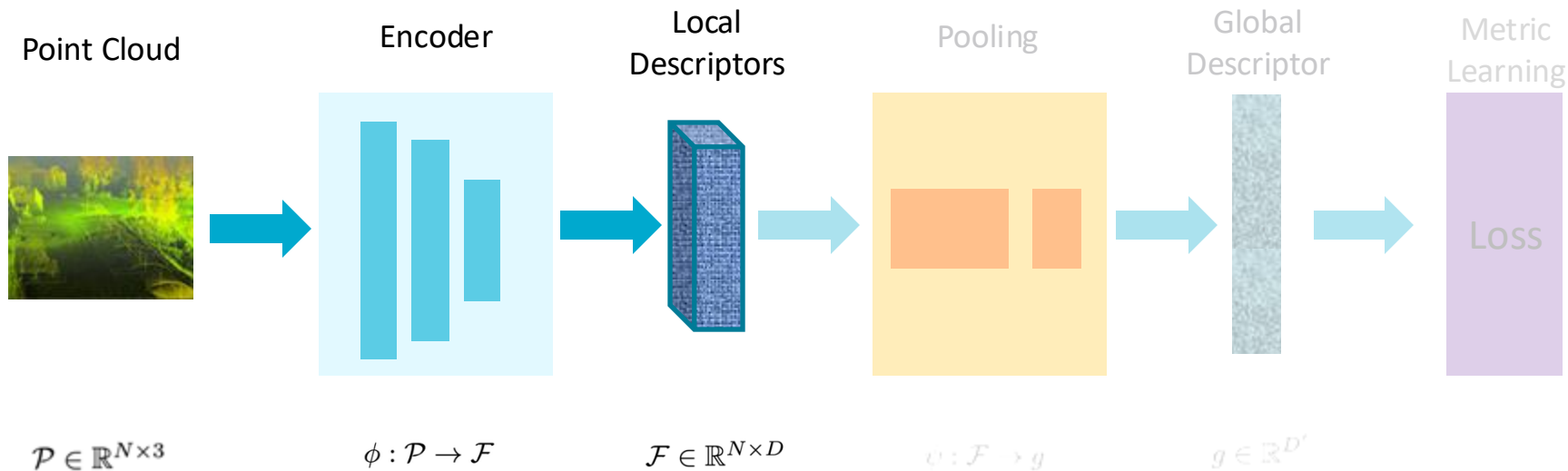
Query Point Cloud



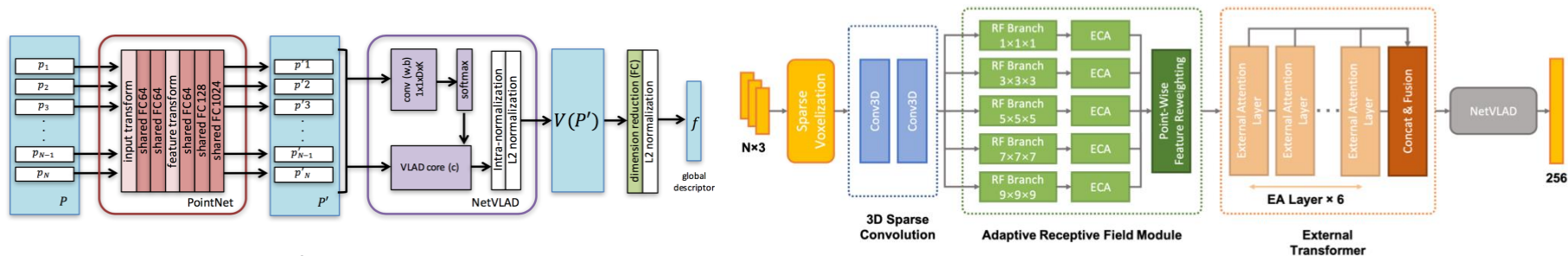
Lidar Place Recognition Network



Place Recognition – Local Descriptors

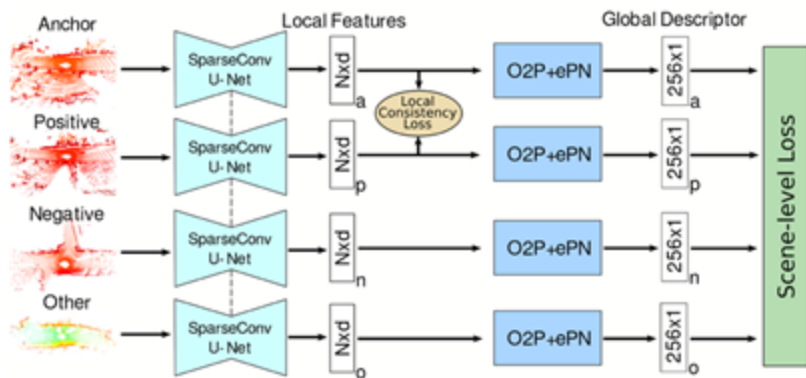


Place Recognition – 3D Encoder Variants

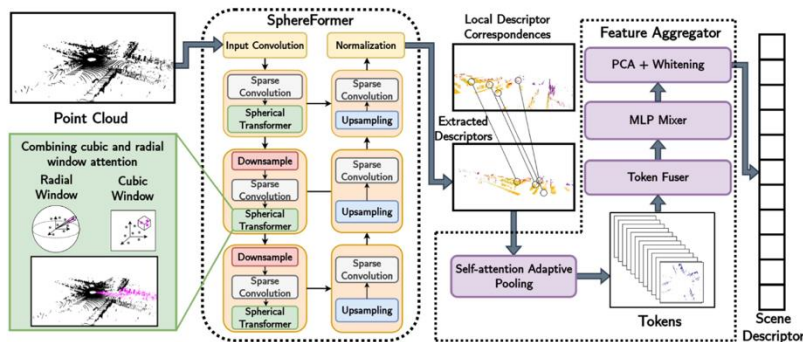


PointNetVLAD
(2018)

TransLoc3D
(2022)

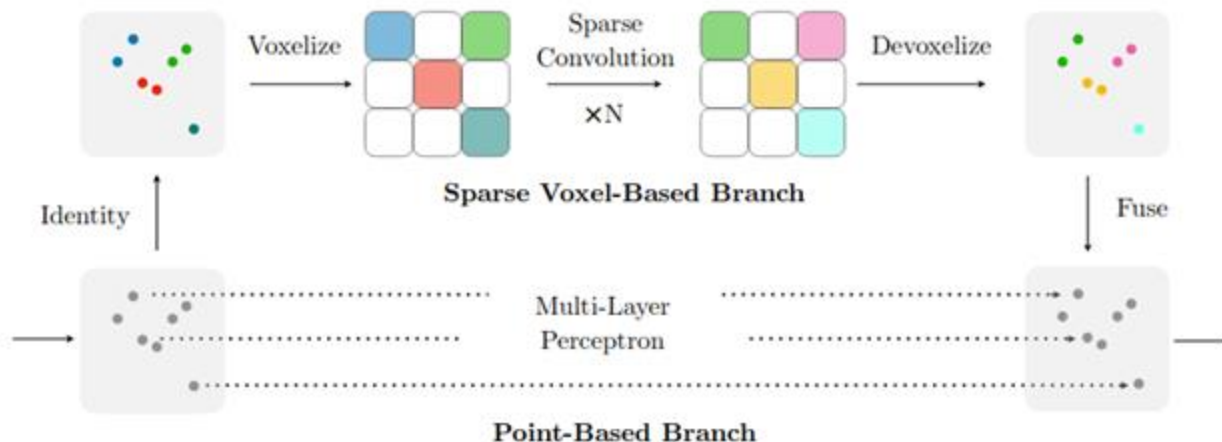


LoGG3D-Net
(2022)



SALSA
(2024)

Place Recognition – Encoder (Sparse ConvNet)

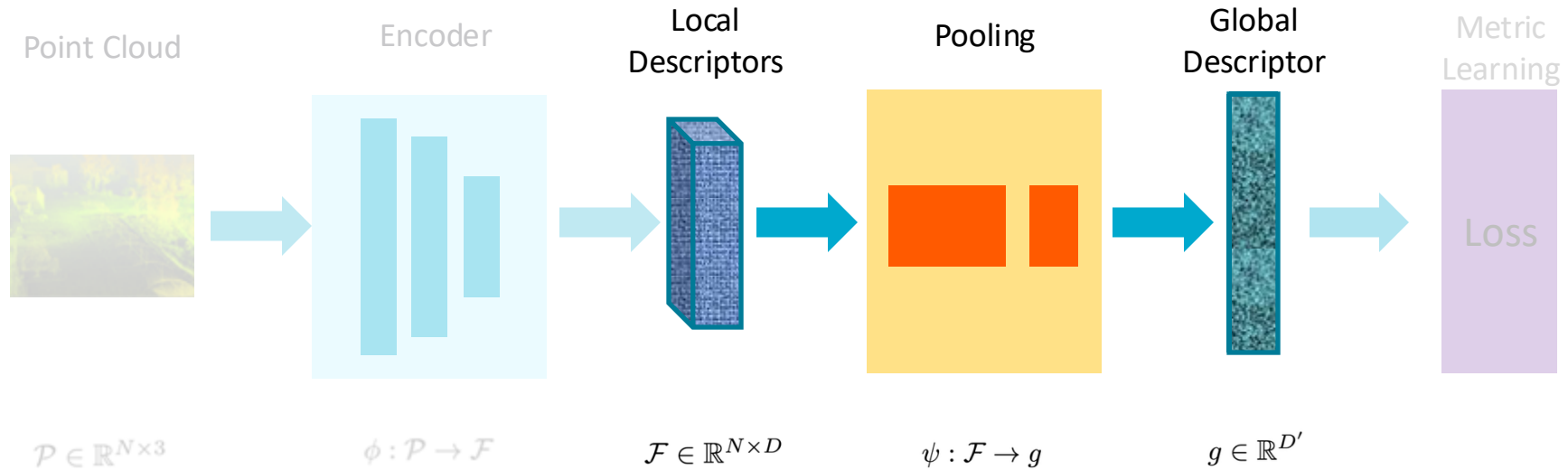


Sparse Point-Voxel Convolution

H. Tang et al., "Searching Efficient 3D Architectures with Sparse Point-Voxel Convolution," ECCV 2020



Place Recognition – Pooling (Permutation Invariance)



Pooling (Aggregation) - First-Order Pooling

Max Pooling

$$g^{(k)} = \max_{j=1,\dots,N} f_j^{(k)}$$

Average Pooling

$$g^{(k)} = \frac{1}{N} \sum_{j=1}^N f_j^{(k)}$$

Generalized Mean Pooling (GeM)

$$g^{(k)} = \left(\frac{1}{N} \sum_{j=1\dots N} \left(f_j^{(k)} \right)^p \right)^{\frac{1}{p}}$$



Q: Why is GeM pooling referred to as “Generalised Mean Pooling”?

Pooling (Aggregation) - First-Order Pooling

Max Pooling

$$g^{(k)} = \lim_{p \rightarrow \infty} \left(\frac{1}{N} \sum_{j=1}^N (f_j^{(k)})^p \right)^{\frac{1}{p}} = \max_{j=1, \dots, N} f_j^{(k)}$$

Average Pooling

$$g^{(k)} = \left(\frac{1}{N} \sum_{j=1}^N (f_j^{(k)})^1 \right)^{\frac{1}{1}} = \frac{1}{N} \sum_{j=1}^N f_j^{(k)}$$

Generalized Mean Pooling (GeM)

$$g^{(k)} = \left(\frac{1}{N} \sum_{j=1 \dots N} (f_j^{(k)})^p \right)^{\frac{1}{p}}$$

Pooling (Aggregation) - First-Order Pooling

Vector of Locally Aggregated Descriptors
(VLAD)

$$g_m^{(k)} = \sum_{j=1}^N a_{j,m} \underbrace{\left(f_j^{(k)} - c_m^{(k)} \right)}$$

hard
assignment

quantization

$$a_{j,m} \in \{0, 1\}$$

Network of Vector of
Locally Aggregated Descriptors (NetVLAD)

$$a_{j,m} = \frac{\exp(-\alpha \|f_j - c_m\|^2)}{\sum_{m'=1}^M \exp(-\alpha \|f_j - c_{m'}\|^2)} = \frac{\exp(\mathbf{w}_m^\top f_j + b_m)}{\sum_{m'=1}^M \exp(\mathbf{w}_{m'}^\top f_j + b_{m'})}$$

soft
assignment

$$a_{j,m} \in [0, 1]$$

$$\begin{aligned} \mathbf{w}_m &= 2\alpha \mathbf{c}_m \\ b_m &= -\alpha \|\mathbf{c}_m\|^2 \end{aligned}$$

Hervé, et al. "Aggregating local descriptors into a compact image representation." CVPR 2010

Arandjelovic, et al. "NetVLAD: CNN architecture for weakly supervised place recognition." CVPR 2016



Pooling (Aggregation)

"Despite different formulas, what do these aggregators fundamentally have in common?"

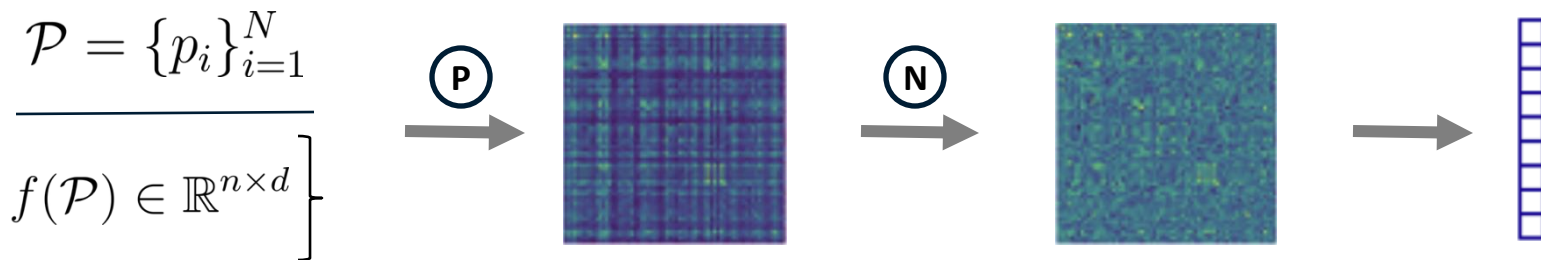


"All you need is a good higher-order pooling"



Pooling (Aggregation) - Second-Order Pooling

With Power Normalization



P

Second-order Pooling

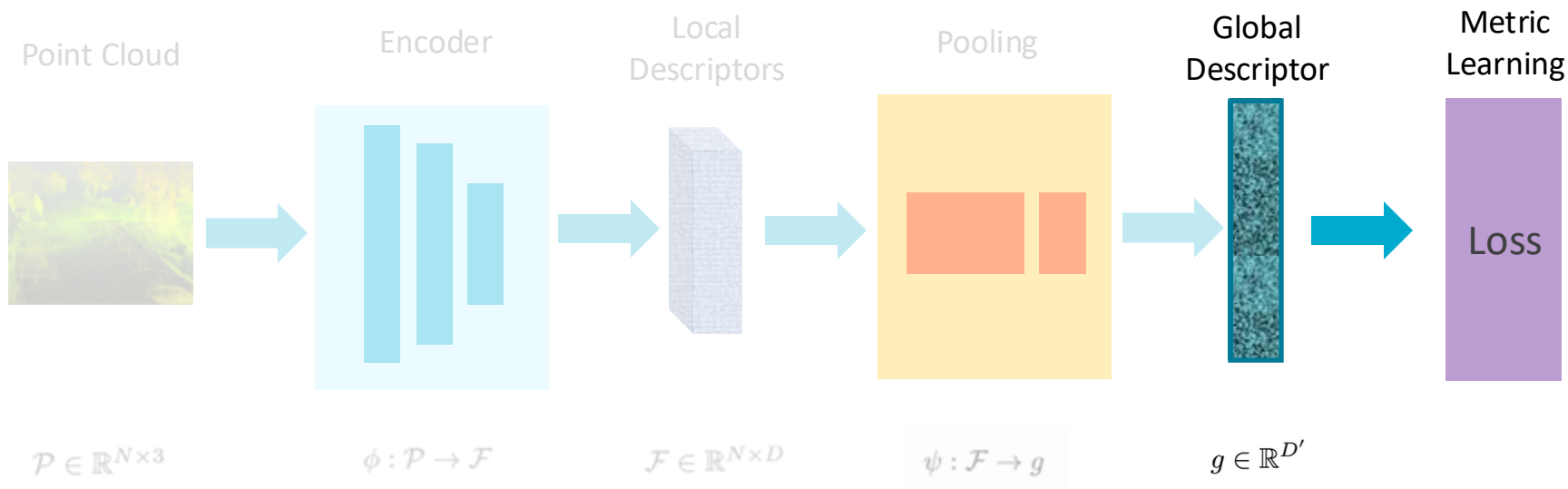
N

Matrix Power Normalization

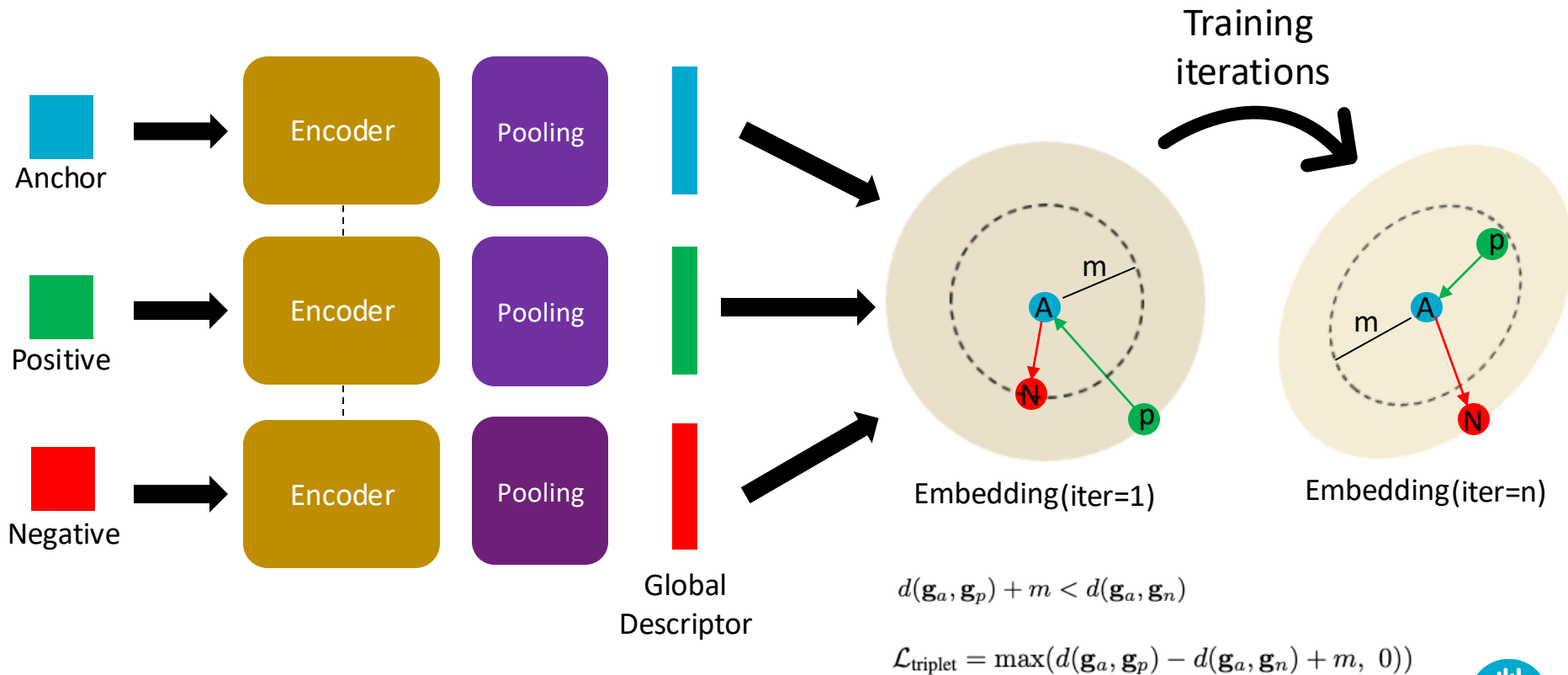
P $f^{o2}(\mathcal{P}) = f(\mathcal{P})^\top f(\mathcal{P}) \in \mathbb{R}^{d \times d}$

N $f^{o2}(\mathcal{P})^\alpha = \mathbf{U} \mathbf{D}^\alpha \mathbf{U}^\top, \mathbf{D}^\alpha = \text{diag}(\lambda_1^\alpha, \lambda_2^\alpha, \dots, \lambda_d^\alpha)$

Place Recognition – Metric Learning



Metric Learning - Triplet Loss



Dash line: shared weights

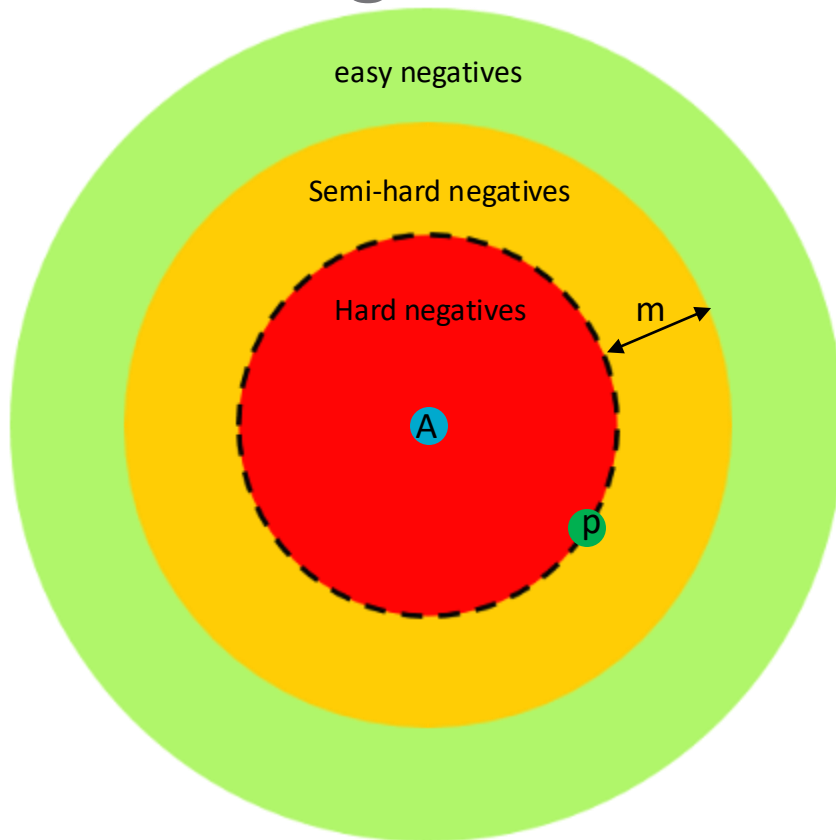
Metric Learning - Triplet Mining

$$\mathcal{L}_{\text{triplet}} = \max(d(\mathbf{g}_a, \mathbf{g}_p) - d(\mathbf{g}_a, \mathbf{g}_n) + m, 0)$$

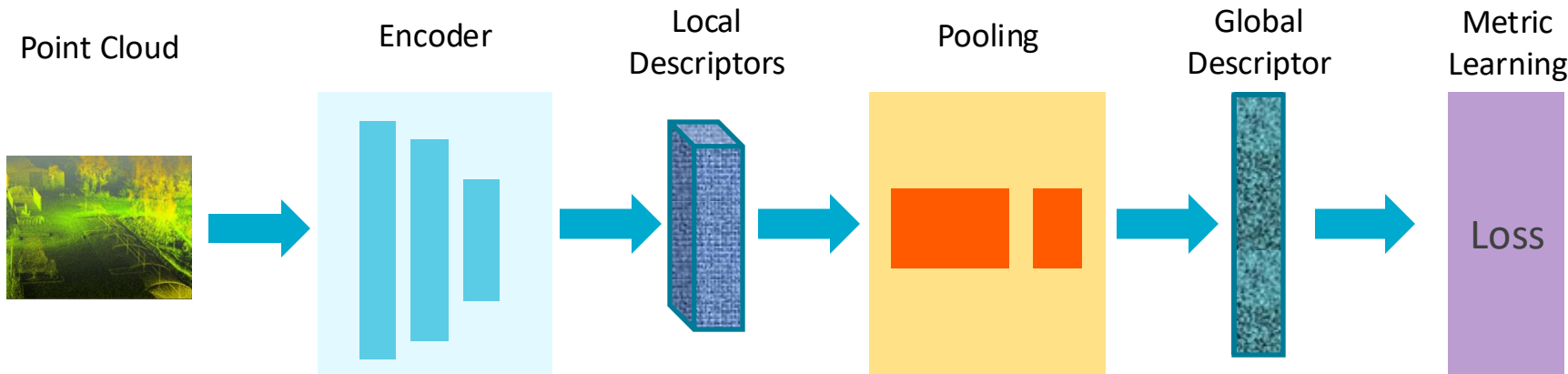
1 Easy Triplets $d(\mathbf{g}_a, \mathbf{g}_p) + m < d(\mathbf{g}_a, \mathbf{g}_n)$

2 Hard Triplets $d(\mathbf{g}_a, \mathbf{g}_n) < d(\mathbf{g}_a, \mathbf{g}_p)$

3 Semi-Hard Triplets $d(\mathbf{g}_a, \mathbf{g}_p) < d(\mathbf{g}_a, \mathbf{g}_n) < d(\mathbf{g}_a, \mathbf{g}_p) + m$



Lidar Place Recognition Network



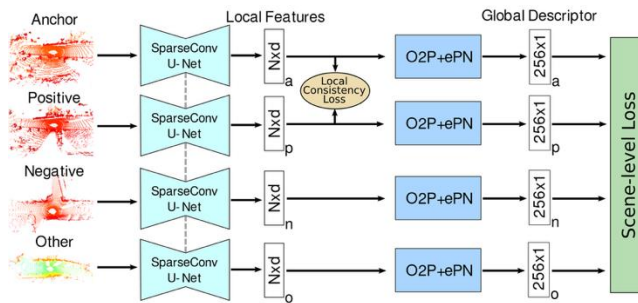
$$\mathcal{P} \in \mathbb{R}^{N \times 3}$$

$$\phi : \mathcal{P} \rightarrow \mathcal{F}$$

$$\mathcal{F} \in \mathbb{R}^{N \times D}$$

$$\psi : \mathcal{F} \rightarrow g$$

$$g \in \mathbb{R}^{D'}$$



Vidanapathirana et. al., "LoGG3D-Net: Locally Guided Global Descriptor Learning for 3D Place Recognition" (ICRA 2022)



Robot Learning – Lidar Place Recognition

Urban, on-road environments

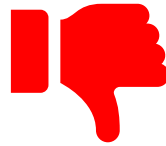
	MulRan Sejong				Apollo-SouthBay				KITTI				Average			
	5 m		20 m		5 m		20 m		5 m		20 m		5 m		20 m	
	R1	R5	R1	R5	R1	R5	R1	R5	R1	R5	R1	R5	R1	R5	R1	R5
MinkLoc3D* [15]	82.3	94.4	92.1	96.6	77.2	93.8	95.0	98.3	95.7	96.9	97.7	98.0	85.0	95.0	94.9	97.6
DiSCO* [35]	94.0	97.5	95.8	98.1	95.1	96.6	95.4	97.0	90.7	91.3	92.3	94.5	93.2	95.1	94.5	96.5
DH3D* [4]	32.4	56.2	58.9	75.5	25.3	49.6	50.4	70.7	75.5	91.1	86.8	96.1	44.4	65.6	65.3	80.7
ScanContext [10]	86.1	89.6	88.5	91.5	91.0	91.0	92.1	92.4	96.3	97.4	96.3	97.6	91.1	92.6	92.3	93.8
Locus [12]	67.0	75.8	70.6	74.6	85.5	95.5	94.9	98.1	99.0	99.7	99.8	100	83.8	90.3	88.4	90.9
EgoNN [6]	98.3	99.9	99.6	99.9	95.7	97.7	96.3	98.2	97.4	98.2	97.9	98.7	97.1	98.6	97.9	98.9
LCDNet [5]	63.1	82.0	85.8	92.1	64.7	81.7	87.4	92.1	96.9	99.4	99.5	99.8	74.9	87.7	90.9	94.6
LoGG3D-Net [18]	97.7	99.7	98.8	99.8	95.2	98.4	98.2	<u>99.2</u>	99.7	99.8	100	100	97.5	<u>99.3</u>	99.0	<u>99.6</u>
EgoNN + SGV	99.1	99.9	99.6	100	<u>98.3</u>	<u>98.7</u>	<u>99.1</u>	<u>99.2</u>	99.2	99.4	99.7	99.7	<u>98.8</u>	<u>99.3</u>	<u>99.4</u>	<u>99.6</u>
LCDNet + SGV	90.8	92.4	94.3	95.8	90.0	90.8	95.2	95.6	99.7	99.7	100	100	93.5	94.3	96.5	97.1
LoGG3D-Net + SGV	<u>99.0</u>	99.9	99.5	100	98.6	99.3	99.6	99.6	99.7	99.8	100	100	99.1	99.6	99.7	99.8

Vidanapathirana et. al., "Spectral Geometric Verification: Re-Ranking Point Cloud Retrieval for Metric Localization" (RAL 2023)

<https://github.com/csiro-robotics/SpectralGV>



Large-scale Lidar place recognition is solved!



Challenges for Robot Learning Methods

Robot Learning – Domain Gaps (C1)



Moderate:  $\dashrightarrow$ 

Method	KITTI→ALITA			KITTI→NCLT			MulRan→ALITA			MulRan→NCLT			Average		
	R@1	R@5	R@1%	R@1	R@5	R@1%	R@1	R@5	R@1%	R@1	R@5	R@1%	R@1	R@5	R@1%
ScanContext [12]	56.03	77.07	59.59	64.74	69.54	74.88	56.03	77.07	59.59	64.74	69.54	74.88	60.39	73.31	67.24
MinkLoc3Dv2 [2]	72.74	92.75	77.44	54.58	71.38	84.42	70.21	91.05	75.08	69.01	86.39	95.70	66.64	85.39	83.16
LoGG3D-Net [3]	59.68	82.76	64.72	41.80	58.43	73.37	71.19	92.57	75.26	33.28	47.12	58.95	52.02	70.41	68.98

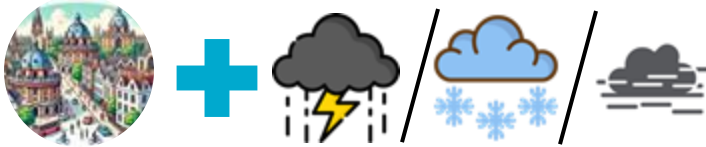
Severe:  $\dashrightarrow$ 

Method	KITTI→Wild-Places(V)			KITTI→Wild-Places(K)			MulRan→Wild-Places(V)			MulRan→Wild-Places(K)			Average		
	R@1	R@5	R@1%	R@1	R@5	R@1%	R@1	R@5	R@1%	R@1	R@5	R@1%	R@1	R@5	R@1%
ScanContext [12]	33.98	48.79	61.56	38.44	53.56	66.16	33.98	48.79	61.56	38.44	53.56	66.16	36.21	51.18	63.86
MinkLoc3Dv2 [2]	5.40	17.15	41.82	6.49	18.79	42.53	11.18	26.47	50.01	14.35	33.32	58.98	9.36	23.93	48.36
LoGG3D-Net [3]	6.00	15.13	32.54	8.55	19.43	37.26	3.91	11.69	27.97	5.16	14.07	30.43	5.91	15.08	32.05

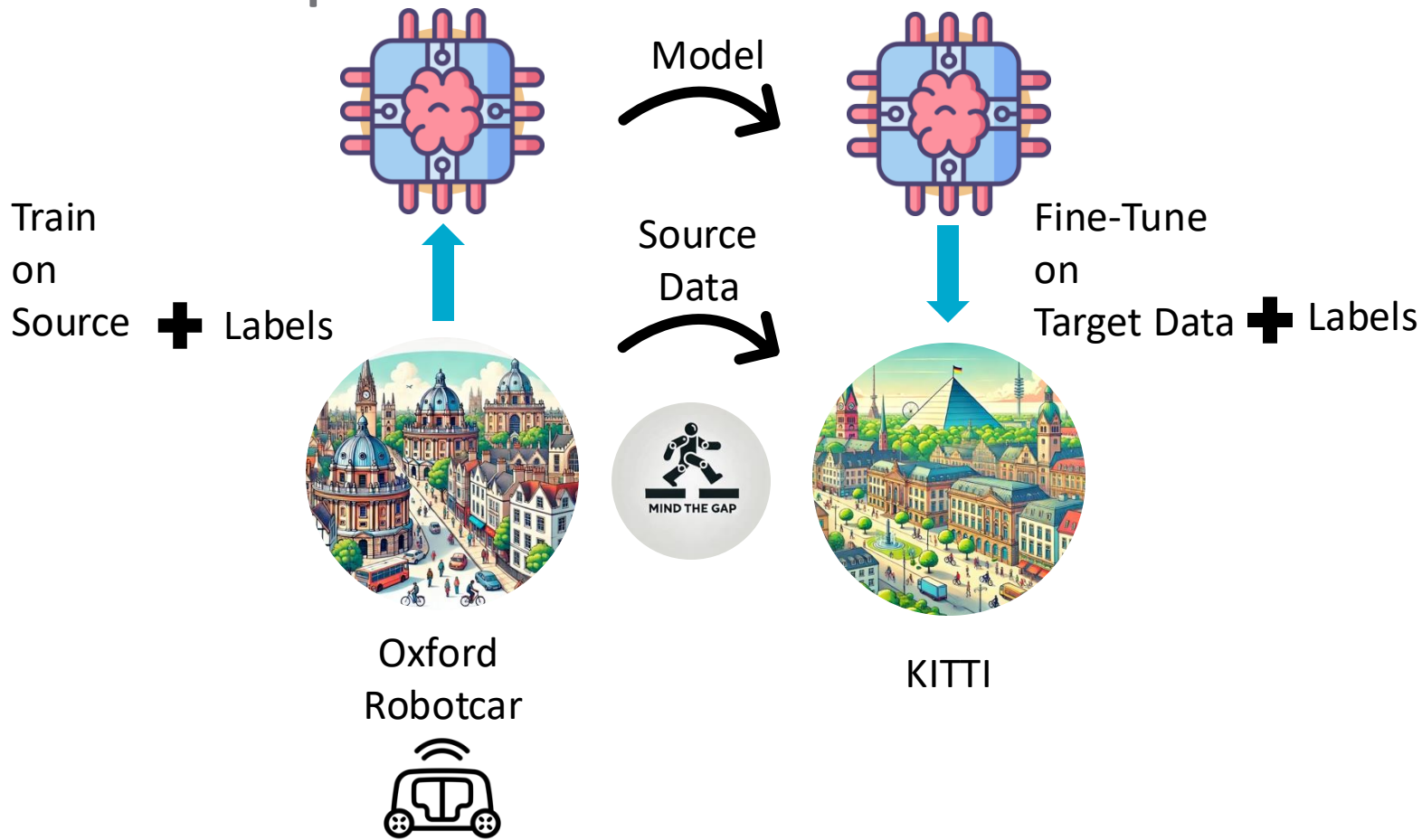
Knights et. al., “GeoAdapt: Self-Supervised Test-Time Adaptation in LiDAR Place Recognition Using Geometric Priors” (RAL 2024)



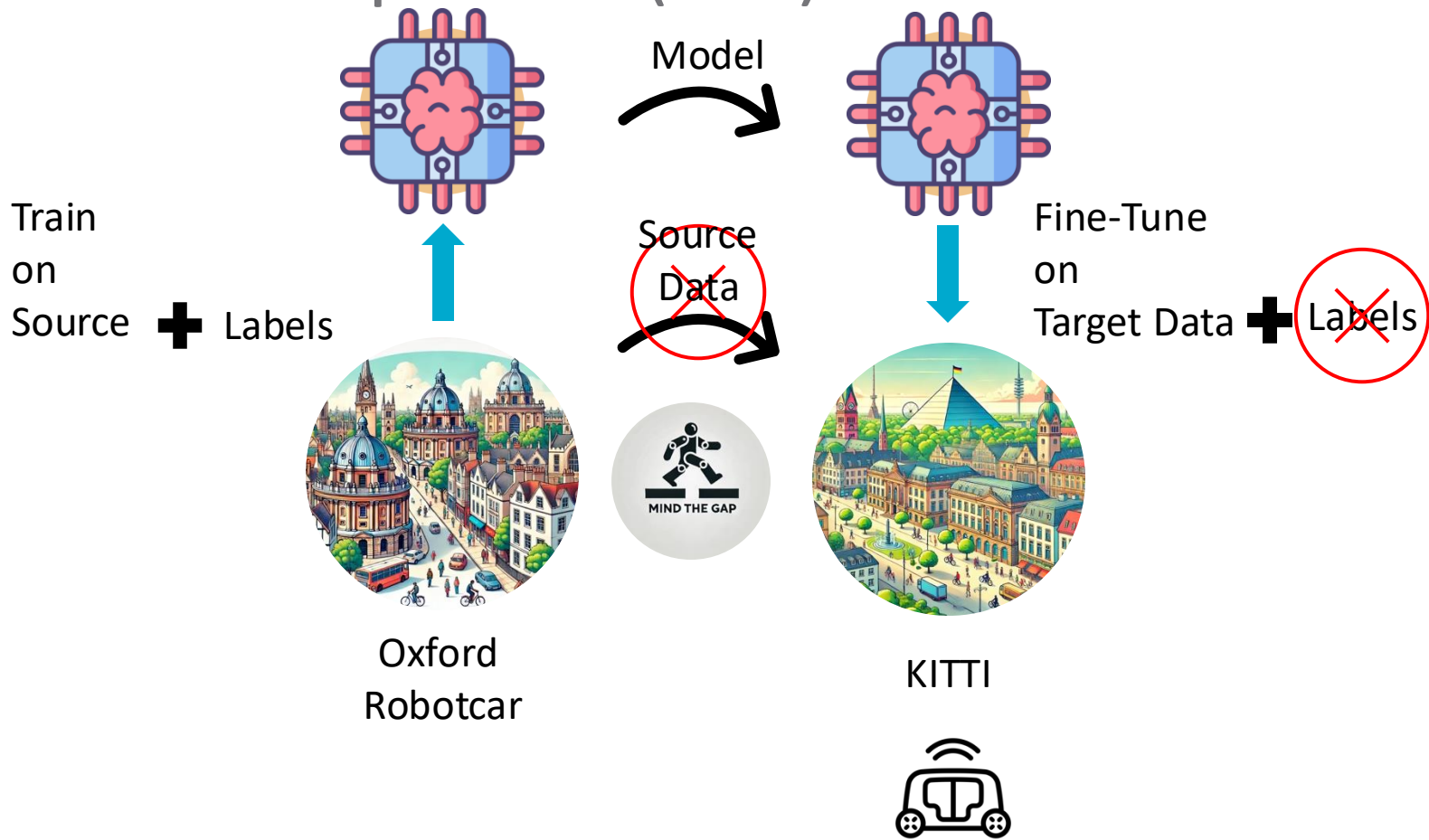
Causes of Domain Shifts

<i>Environmental</i> Domain Shift	
<i>Temporal</i> Domain Shift	
<i>Weather</i> Domain Shift	
<i>Sensor</i> Domain Shift	

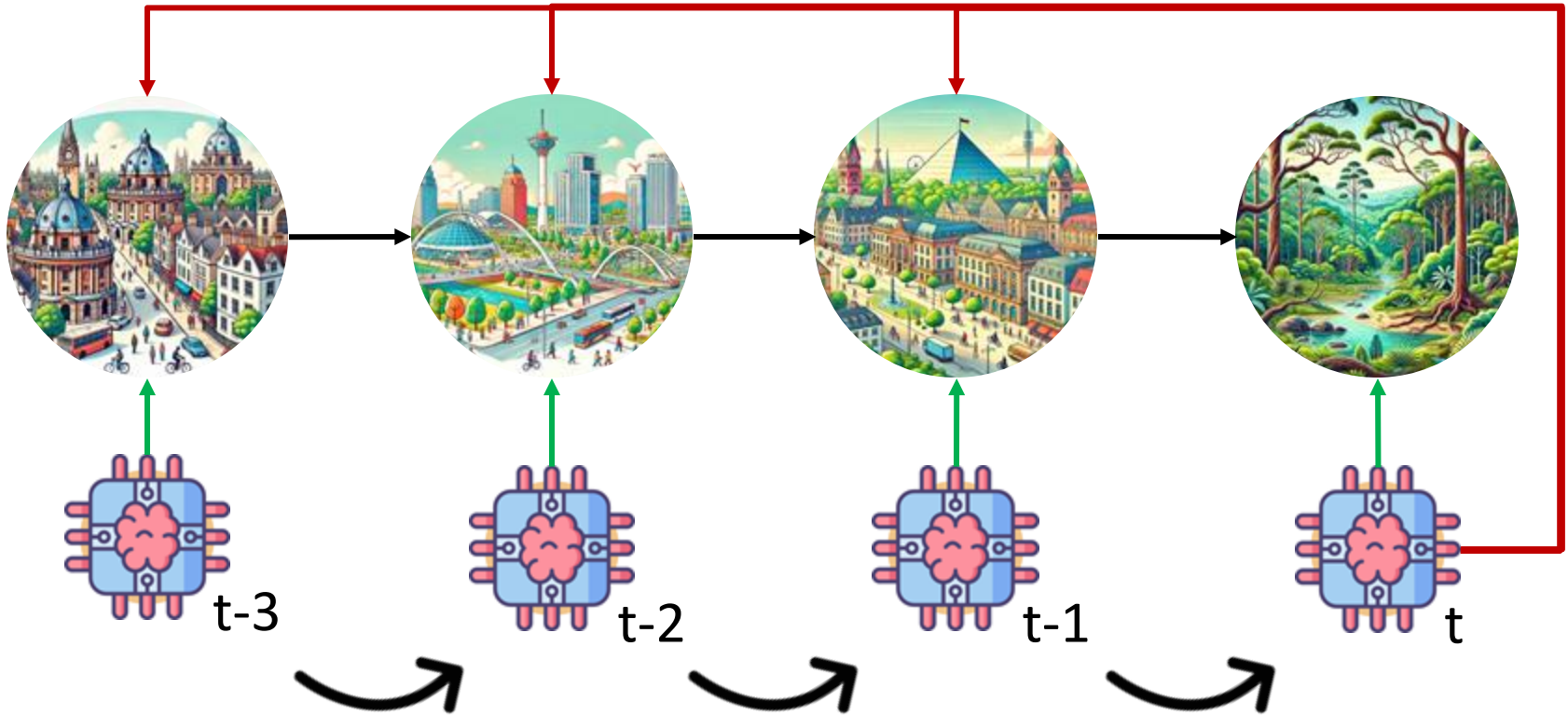
Domain Adaptation



Test-Time Adaptation (TTA)



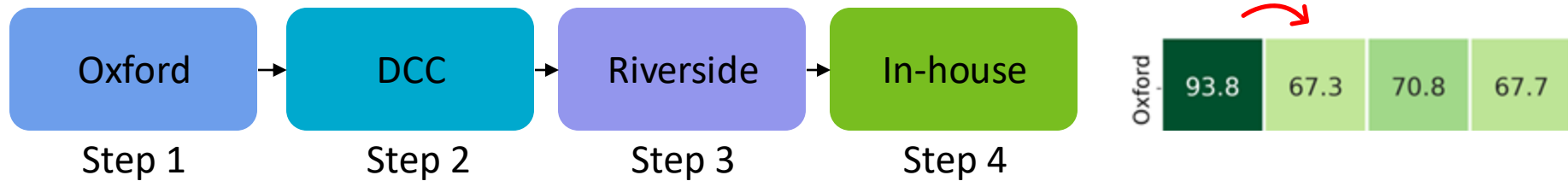
Robot Learning – Catastrophic Forgetting (C2)



- Test on **new domains**

- Test on **old domains**

Robot Learning – Catastrophic Forgetting (C2)



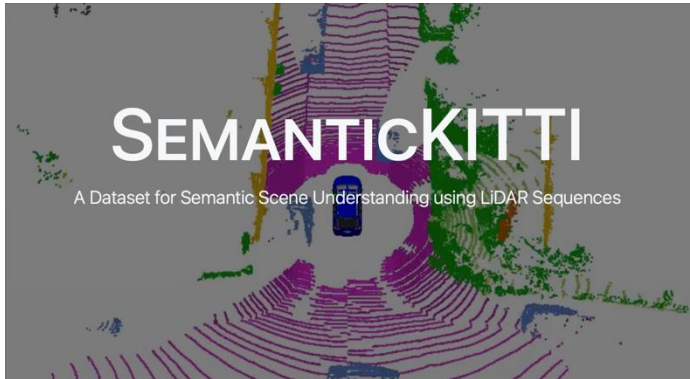
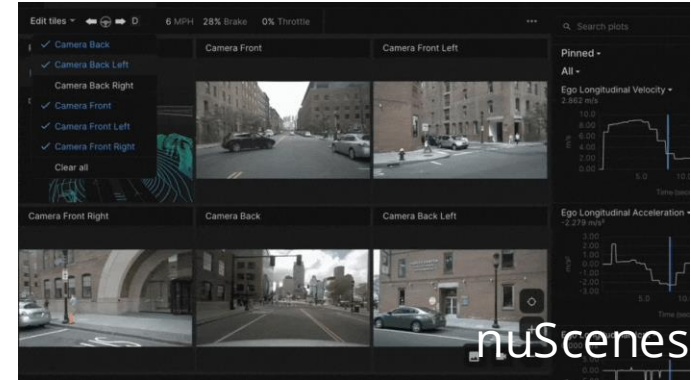
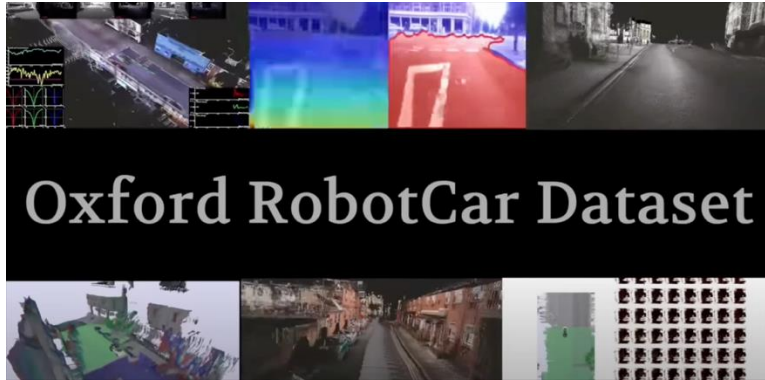
- Fine-tuning a model on new domains results in degradation of performance on old domains
- This phenomenon is known as *Catastrophic Forgetting*

Step 1 Step 2 Step 3 Step 4

Recall@1

In the era of foundation models, the demand for large-scale benchmarks has never been more critical!

Robot Learning – Datasets and Benchmarks (C3)



Open X-Embodiment: Robotic Learning Datasets and RT-X Models

Open X-Embodiment Collaboration

(hover to display full author list)



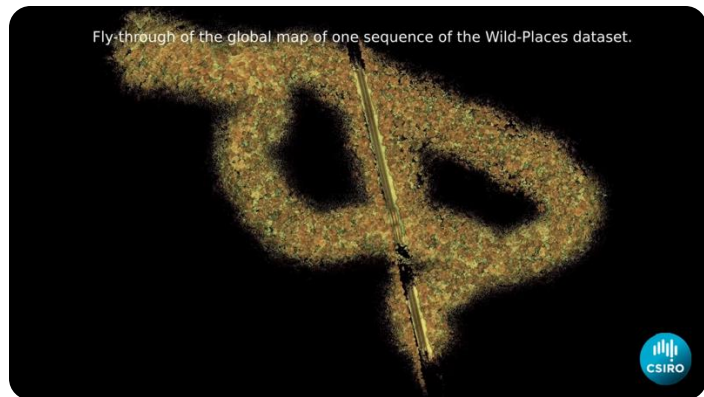
<https://robotics-transformer-x.github.io>



Long-term Benchmarks in Natural Environments

WildPlaces Benchmark Dataset

<https://csiro-robotics.github.io/Wild-Places>



63K lidar frames



Open Data
Open Models

WildScenes Benchmark Dataset

<https://csiro-robotics.github.io/WildScenes>



12K lidar frames
9K image frames



Summary of Today's Lecture

- 3D Understanding Tasks
- Common Sensors, Scene Representations
- Sparse Convolution versus Dense Convolution
- Lidar Place Recognition
- Challenge for Robot Learning Methods
 - Domain Gaps
 - Catastrophic Forgetting
 - Benchmarks and Datasets

Next Robot Learning Lecture D3: Robotic Foundation Models
Friday 1:30pm

Thank you

Peyman Moghadam



Code on Github

<https://github.com/csiro-robotics>

peyman.moghadam@csiro.au

peyman.moghadam@qut.edu.au

